



دانشگاه صنعتی امیر کبیر
(پلی تکنیک تهران)

Computer Architecture

Chapter 2

Basic Computer Design – part 1

Afarghadan@aut.ac.ir

عظیم فرقدان 

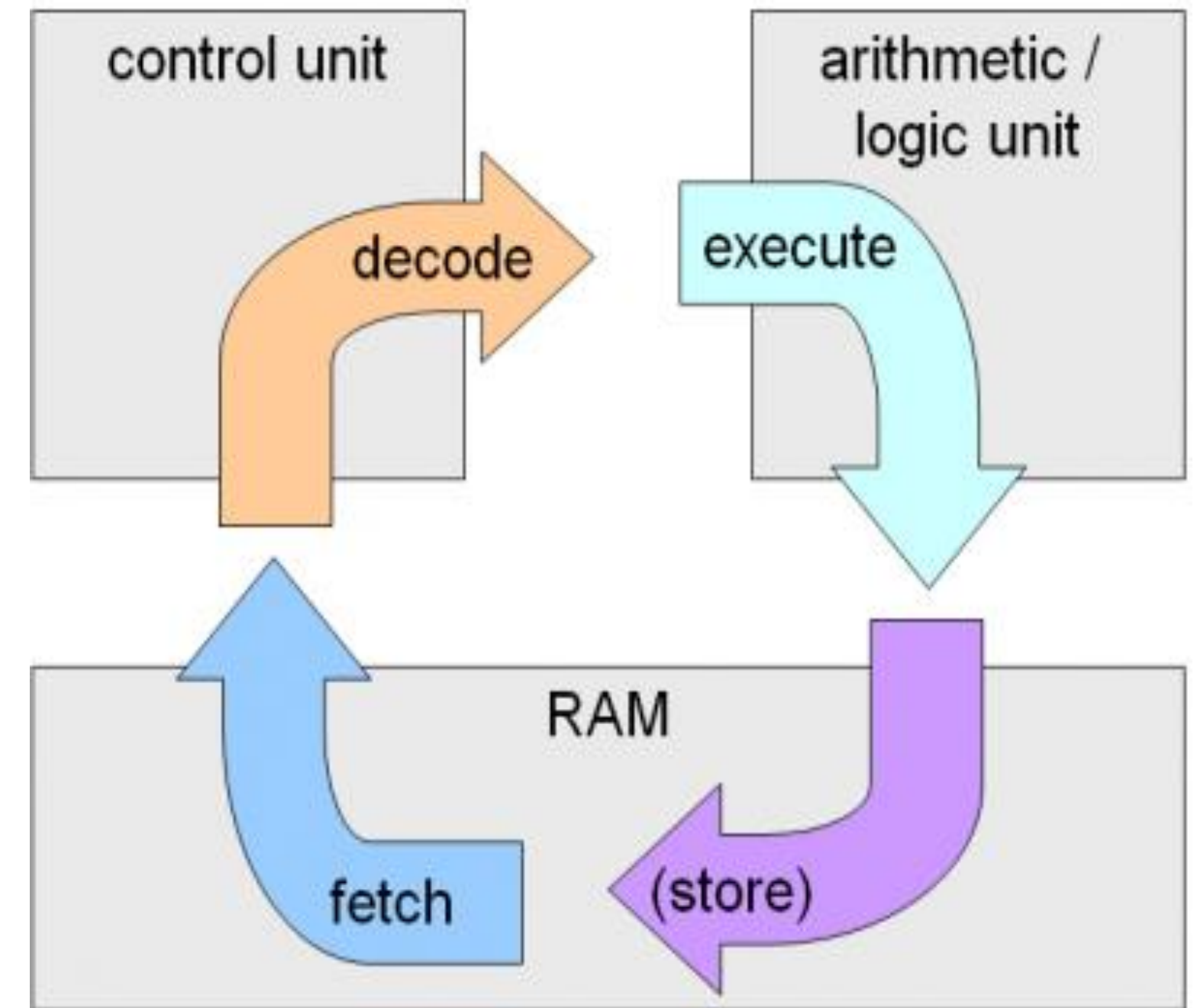
بهمن ۱۴۰۳

- 1. MicroOperations**
- 2. Memory**
- 3. Instruction Set Architecture**
- 4. Basic Computer Introduction**
- 5. Computer Registers**
- 6. Register Transfer Language**

- ✓ The user of a computer can control the process by means of a program.
- ✓ A program is a set of instructions that specify the **operations**, **operands**, and the **sequence** by which processing has to occur.
- ✓ A computer instruction is **a binary code** that specifies a sequence of **microoperations** for the computer.
- ✓ **Instruction codes** together with **data** are stored in **memory**.

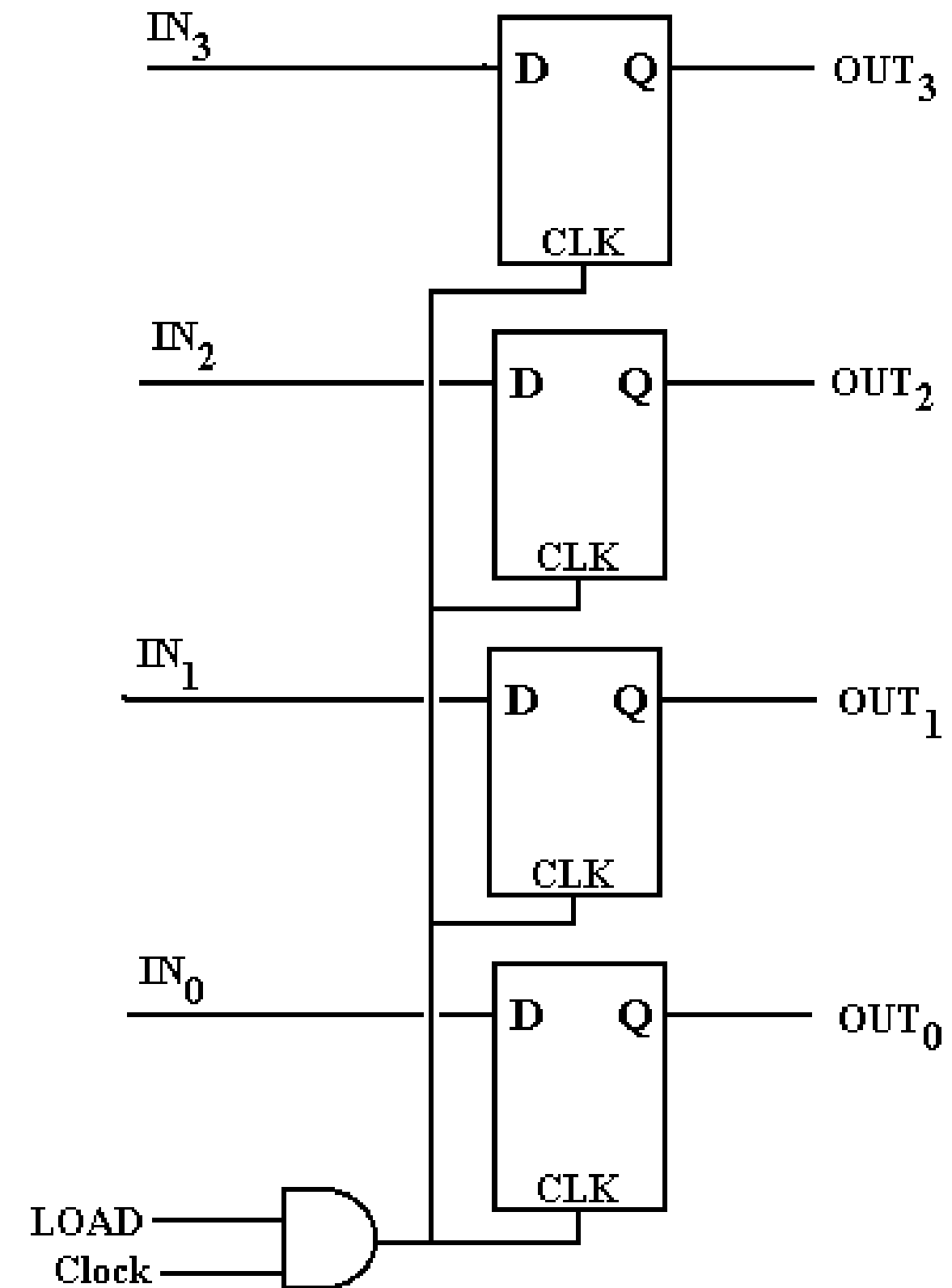
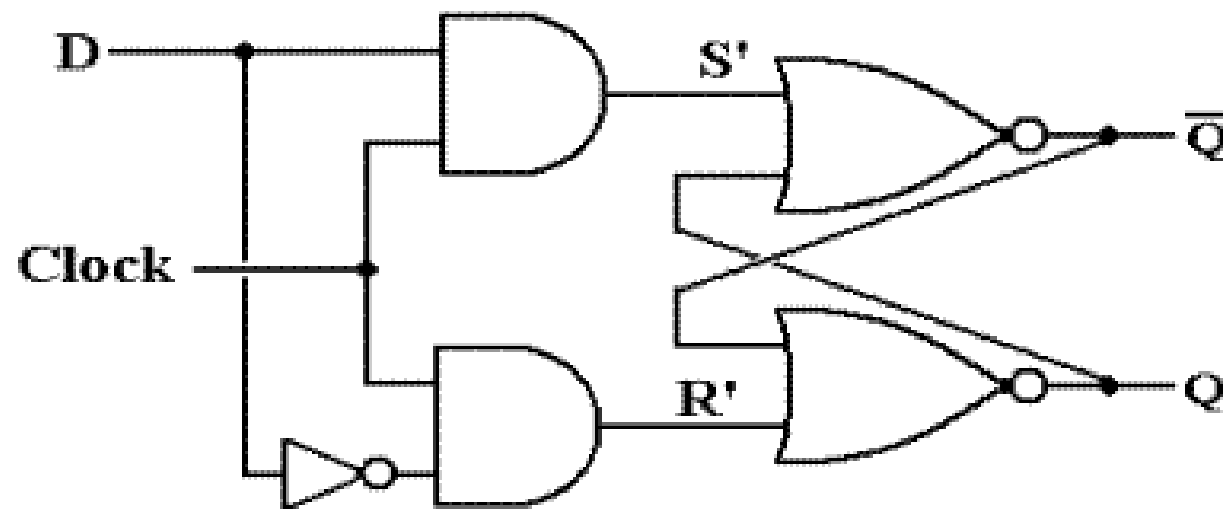
- **Instruction** is a binary code that tells the computer to perform a specific operation. An **operation** is a part of an instruction stored in the memory.
- To perform the operation, **a sequence of control signals** are issued to **initiate a set of microoperations**.
- Operation is called **Macrooperation** because it specifies **a set of microoperations**.

- ✓ Stored program computer
 - ✓ Instructions and data stored in **same memory**
- ✓ Sequential instruction processing
 - ✓ Instructions fetched one by one from memory
 - ✓ Program counter incremented in each step



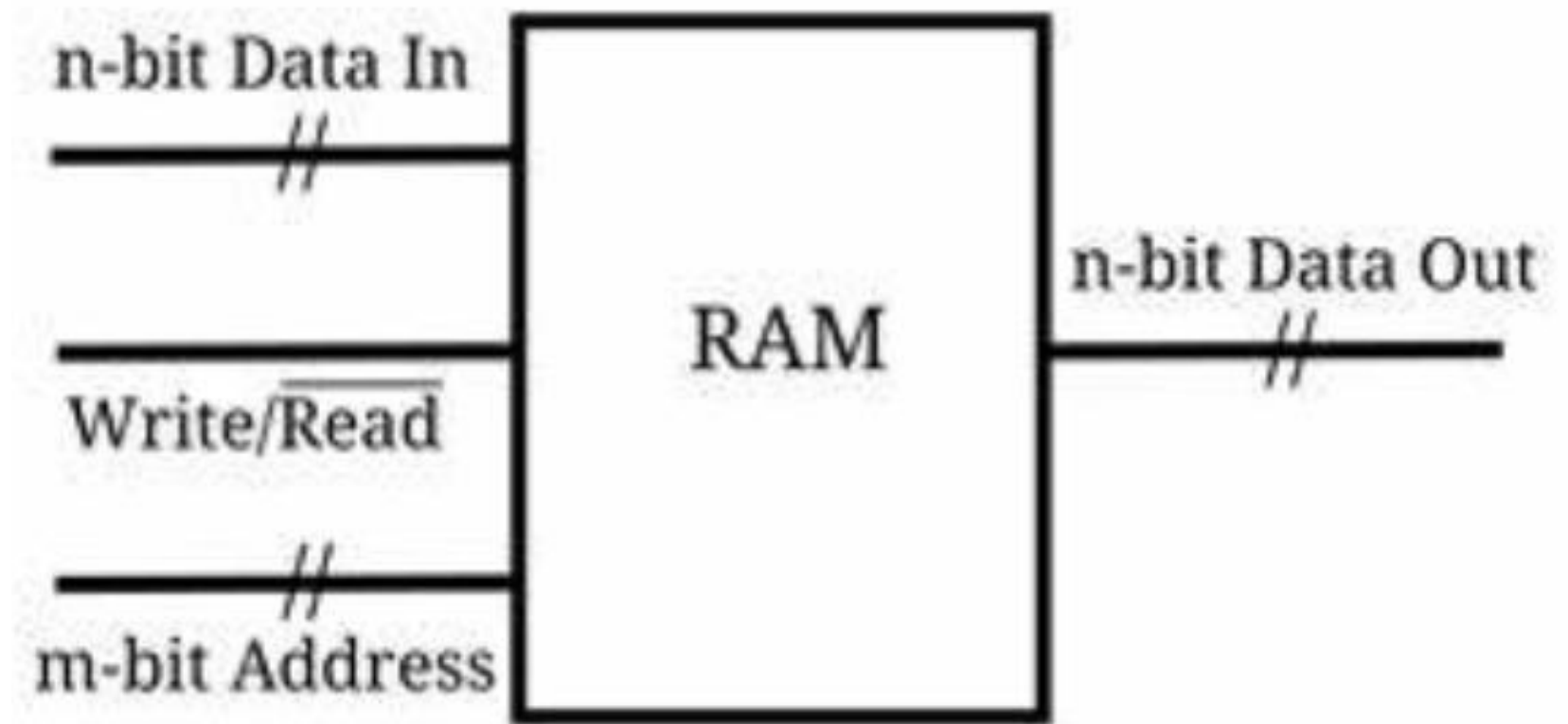
Register

- A **register** is a name for a **collection of flip-flops** used to hold data
- Registers are **the fastest place to hold data** in a computer, so we want to use them as much as possible.

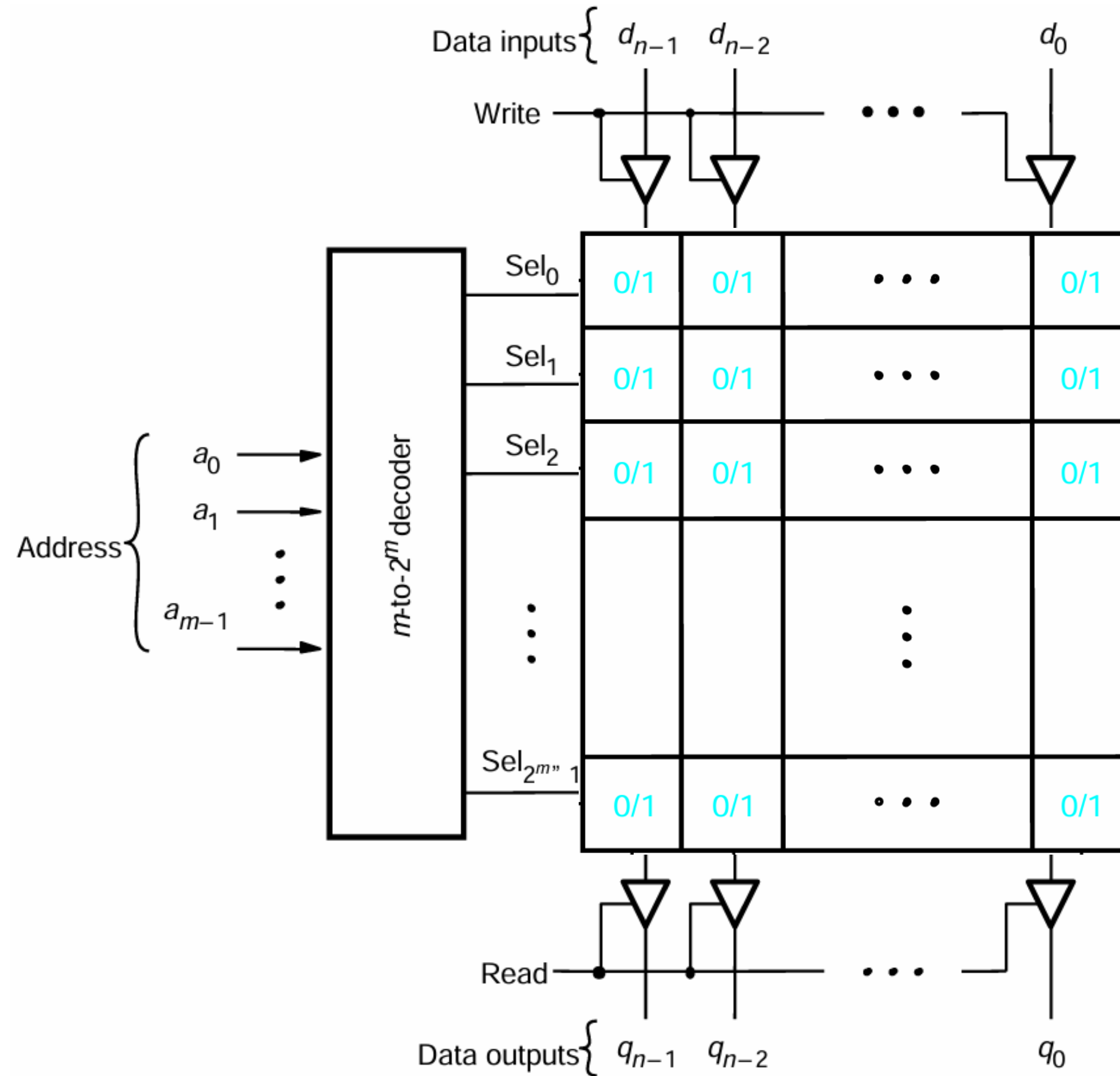


Random Access Memory

- ✓ More efficient than registers for storing large amounts of data
- ✓ Can read and write to **RAM**
- ✓ Addressable memory
- ✓ Synchronous or asynchronous
- ✓ **SRAM** dimensions are:
 - ✓ (number of words) x (bits per word)
- ✓ Address is m bits, data is n bits
 - ✓ 2^m x **n-bit** RAM
- ✓ Write or Read



Random Access Memory



- the **instructions are the words** of a computer's language and the **instruction set is its vocabulary**
- The only way that you **can talk to your machine** is through the ISA.
- ISA is an **interface** between the **hardware and software**.
- The **instruction set architecture** is the specification of **what the computer can do** and **the machine** has to be fabricated in such a way that it will **execute** whatever has been specified in **your ISA**.
- ISA is the **starting point for designing a new machine**.

▪ uArch (microarchitecture)

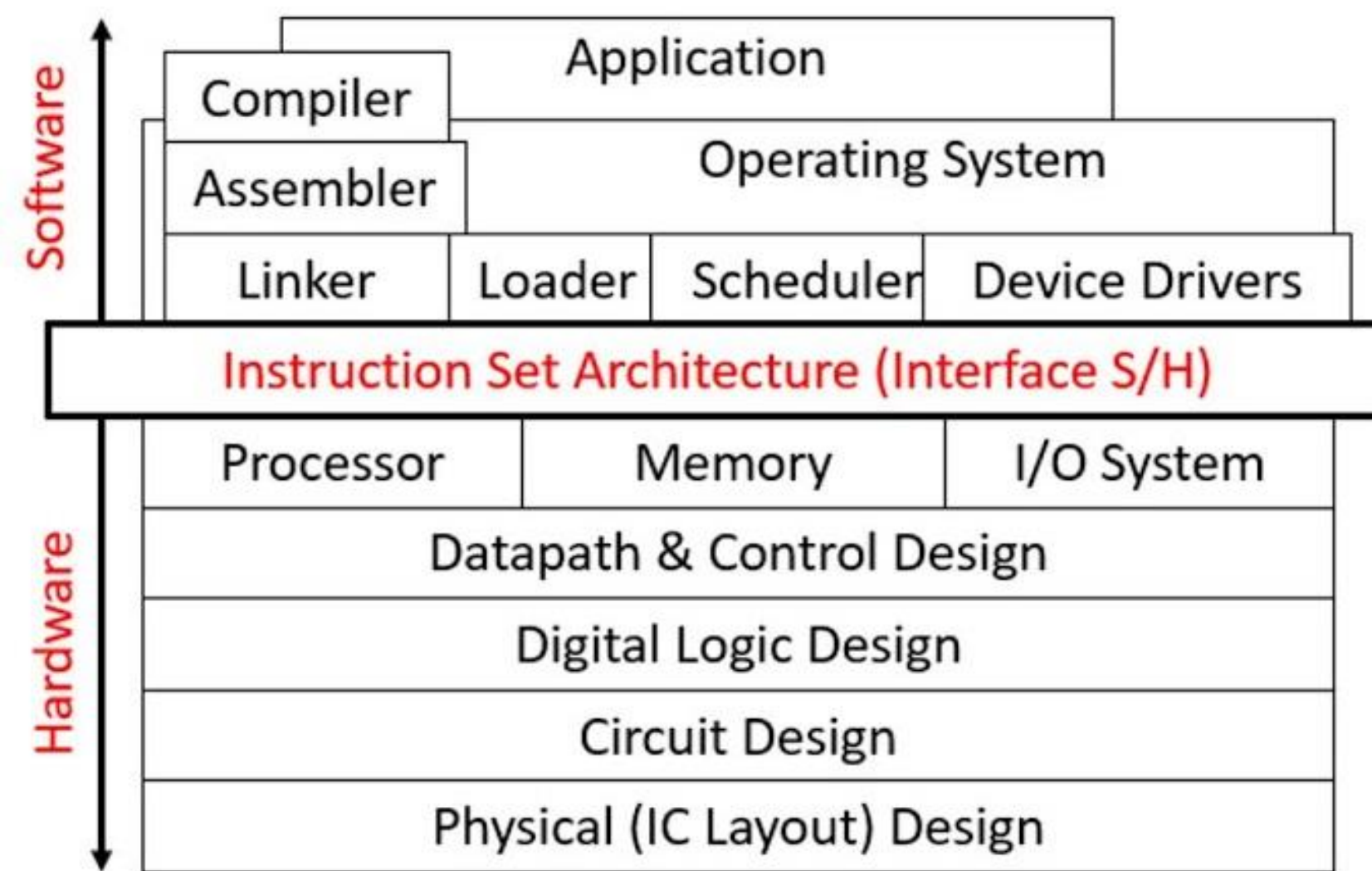
- The **internal structure of the processor** which specifies the way an ISA is implemented
- ISA can be implemented with different uArch

▪ Computer Architecture

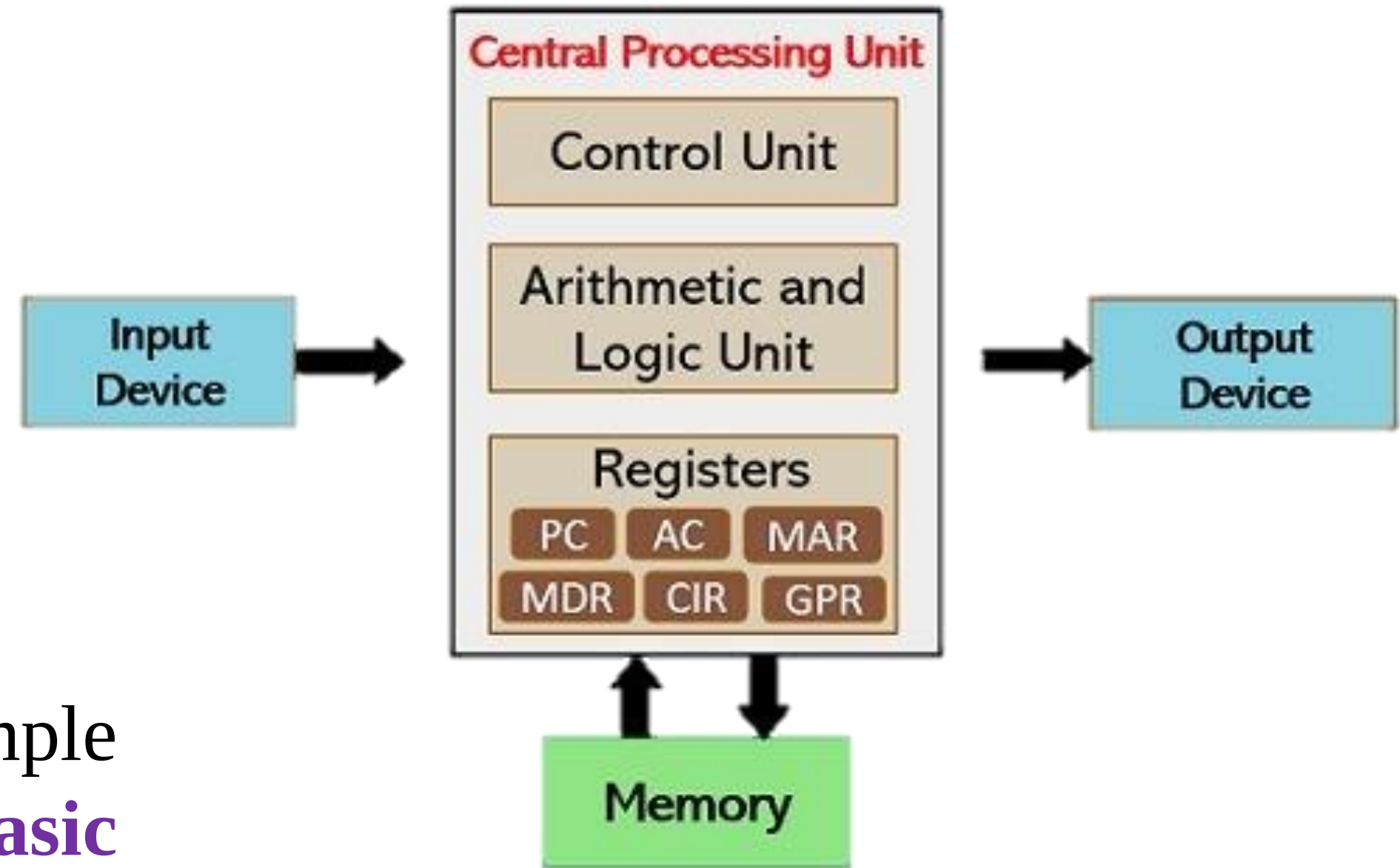
- ISA + uArch

▪ ISA specifies:

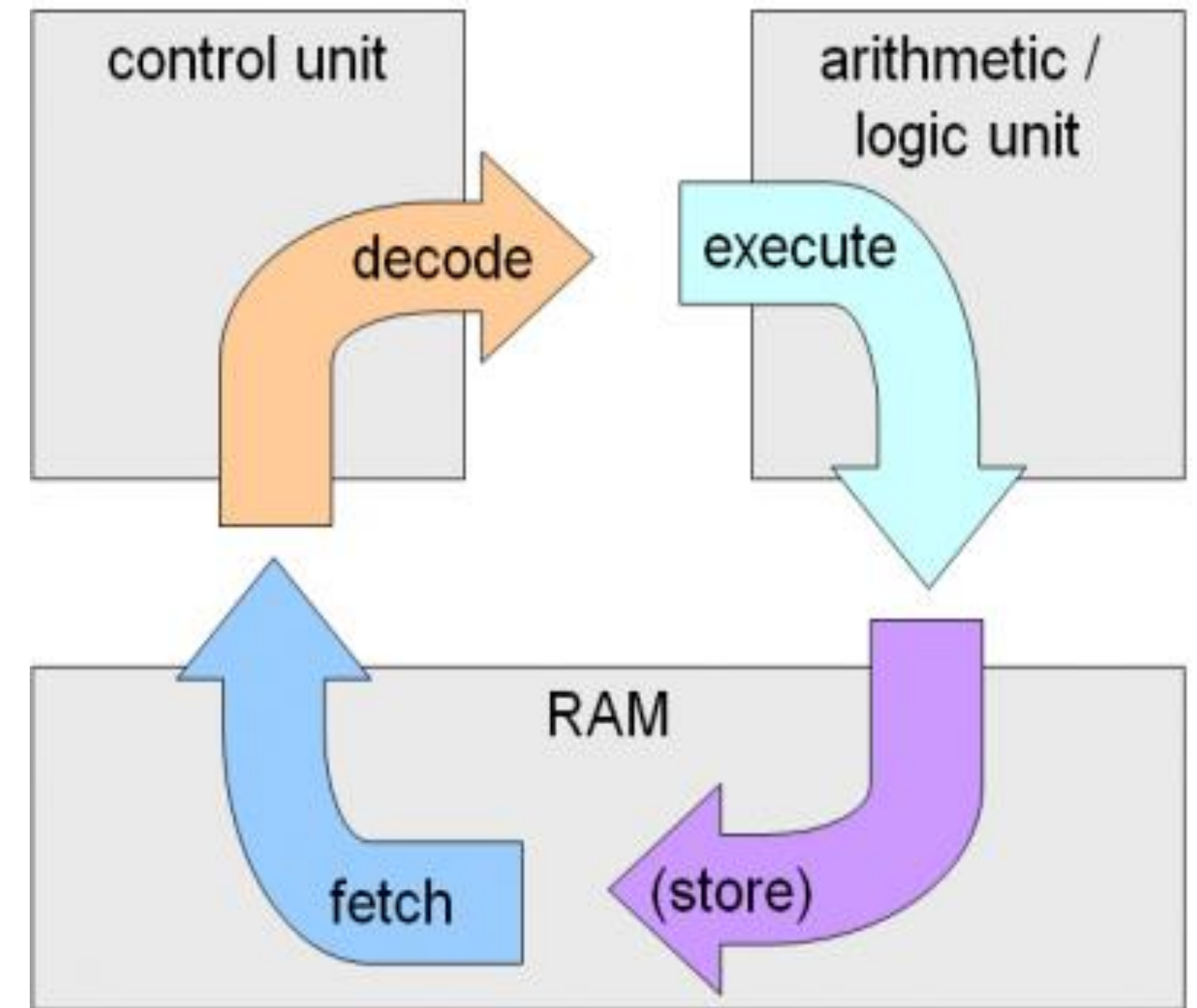
- Instruction format and Instruction length
- Data types
- Registers
- Memory access and addressing
- Addressing modes



- Modern processors are very complex.
- They contain:
 - Many registers
 - Multiple arithmetic units
 - The ability to **pipeline**
 - ...
- M. Morris Mano introduces a simple processor model called the **Basic Computer**.

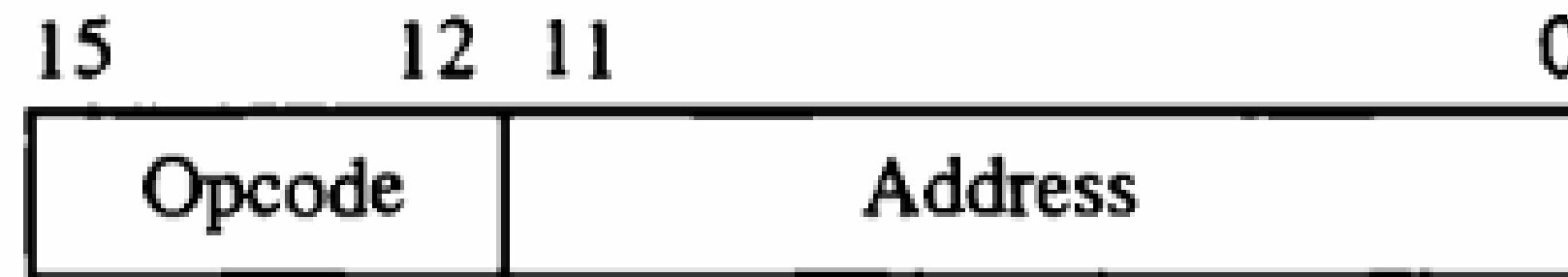


- ✓ Stored program computer
 - ✓ Instructions and data stored in **same memory**
- ✓ Sequential instruction processing
 - ✓ Instructions fetched one by one from memory
 - ✓ Program counter incremented in each step

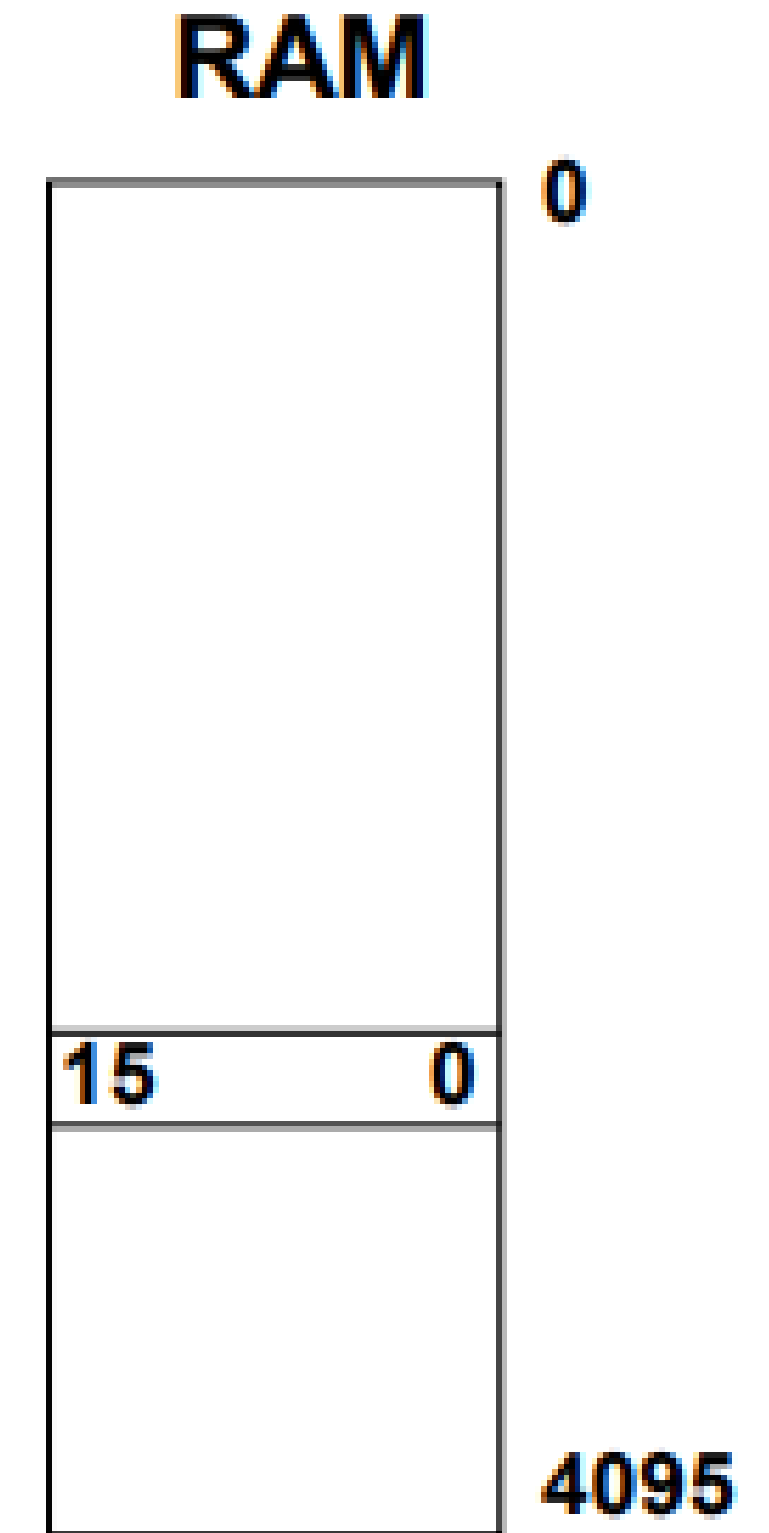


Instruction Format

- The basic computer's memory has **4096 words**.
 - **4096** = 2^{12} , so it takes **12 bits** to select a word in the memory.
- Each word is **16 bits** long.
- A computer instruction is often divided into two parts
 - An **opcode** (Operation Code) that specifies the operation for that instruction
 - An **address** that specifies the registers or memory address to use for that operation



Instruction format



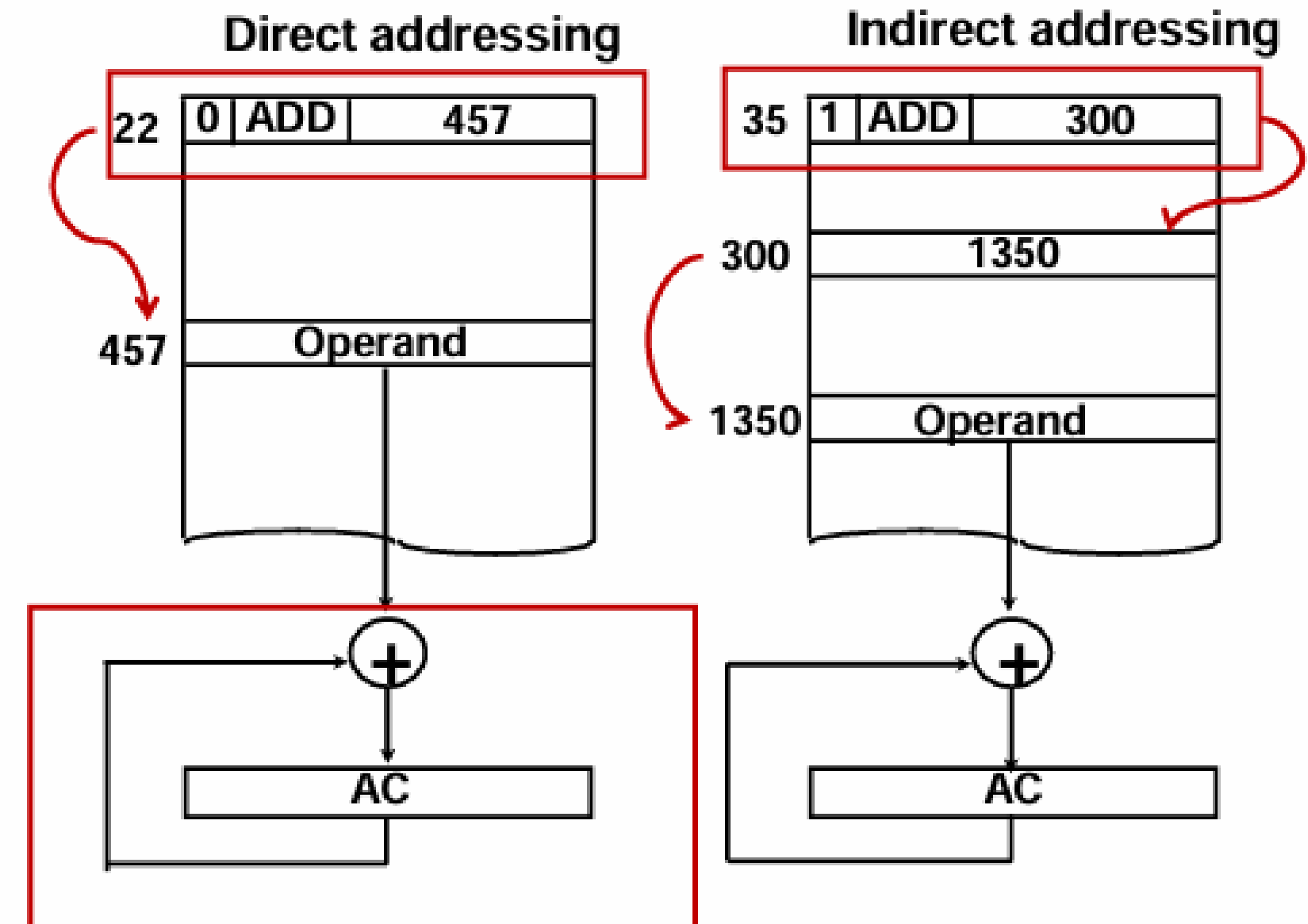
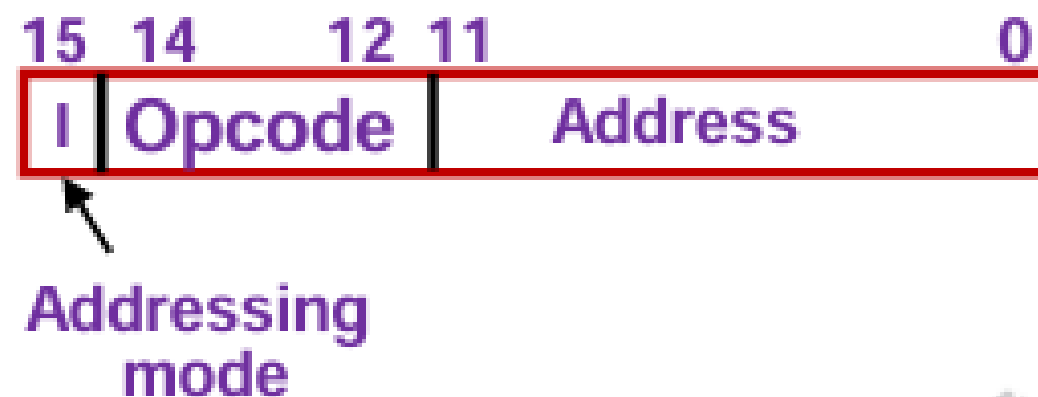
- **Program** is a sequence of (machine) **instructions**.
- (Machine) instruction
 - A group of bits that tell the computer to perform a specific operation (a sequence of **microoperations**).
- The instructions of a program and the needed data are stored in the **memory**.
- The **CPU** reads the instructions from **memory**
- The instruction is placed in an **Instruction Register (IR)**



- **Control unit** translates the instruction in the IR into a sequence of microoperations.

Addressing Modes

- In the basic computer, 15th bit of the instruction specifies the addressing mode (0: direct addressing, 1: indirect addressing)
- The address field of instruction can be either
 - **Direct address**: the memory address of the data
 - **Indirect address**: the memory address of the memory address of the data



- **Effective Address (EA)**
- The address that can be directly used without modification to access an operand for a computation-type instruction, or as the target address for a branch-type instruction

Processor Registers

- The processor has a register called the **Program Counter (PC)** that holds the memory address of the **next instruction** to get.
 - Since the memory in the basic computer only has 4096 locations, the PC only needs 12 bits.



- The processor needs to **keep track of the location** in memory it is addressing
- The **Address Register (AR)** is used for this
 - The AR is a 12-bit register in the basic computer.



- When the operand is found, using either direct or indirect addressing, it is placed in the **Data Register (DR)**
- The processor uses this value as **data** for its operation



- The basic computer has a single **general-purpose** register called the **Accumulator (AC)**



- The significance of a general-purpose register is that it can be **referred** to in instructions.
 - For example: load AC, store AC

- Often a processor will need a scratch register to store intermediate results or other temporary data. In the basic computer this is the **Temporary Register (TR)**

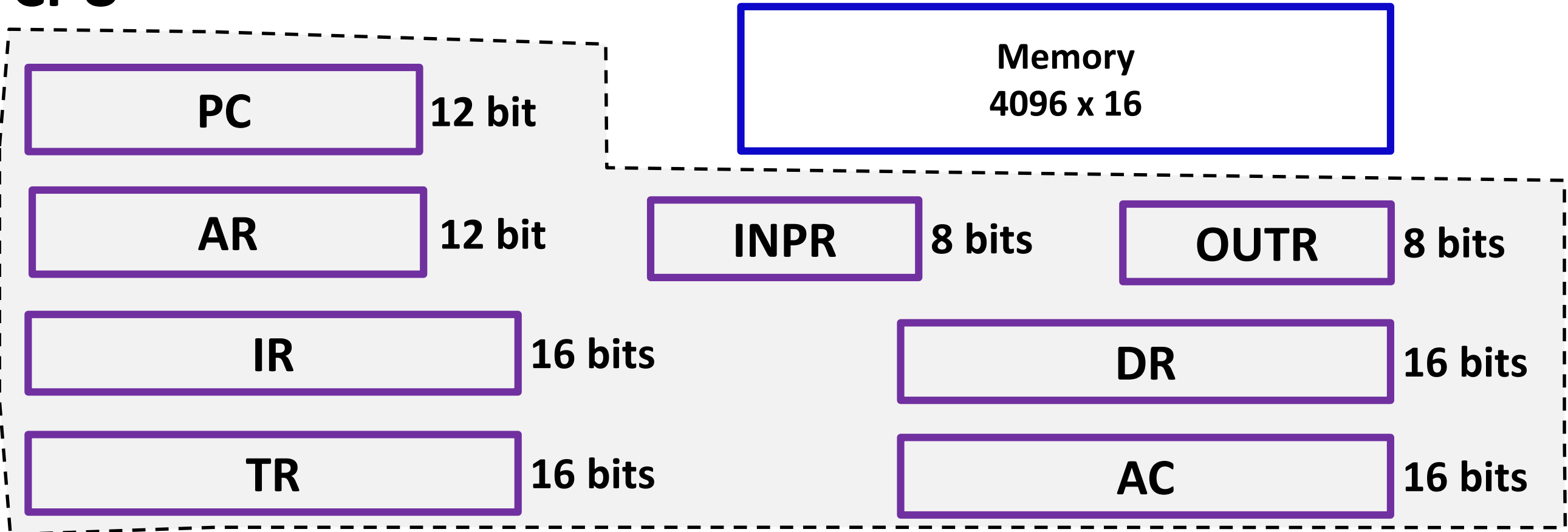


- The basic computer uses a very simple model of input/output (I/O) operations.
 - **Input devices** are considered to **send 8 bits of character data** to the processor
 - The processor can **send 8 bits of character data** to **output devices**
- The **Input Register (INPR)** holds an 8 bit character gotten from an input device.
- The **Output Register (OUTR)** holds an 8 bit character to be send to an output device.



Basic Computer Registers

CPU

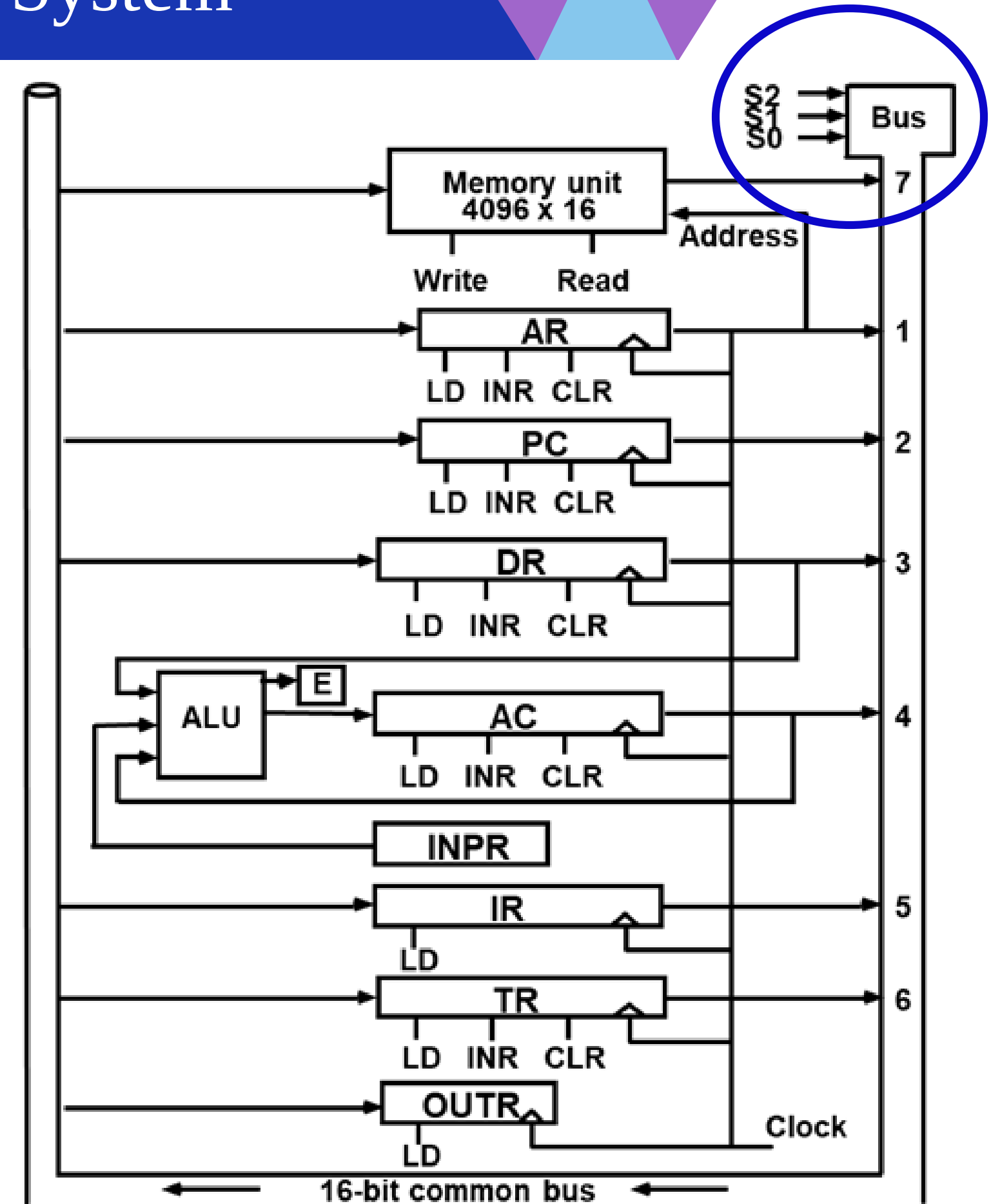


DR	16	Data Register	Holds memory operand
AR	12	Address Register	Holds address for memory
AC	16	Accumulator	Process register
IR	16	Instruction Register	Holds instruction code
PC	12	Program Counter	Holds address of instruction
TR	16	Temporary Register	Holds temporary data
INPR	8	Input Register	Holds input character
OUTR	8	Output Register	Holds output character

Common Bus System

- The registers in the basic computer are connected using a **bus**. This helps **simplify the circuitry** over complete connections between registers.

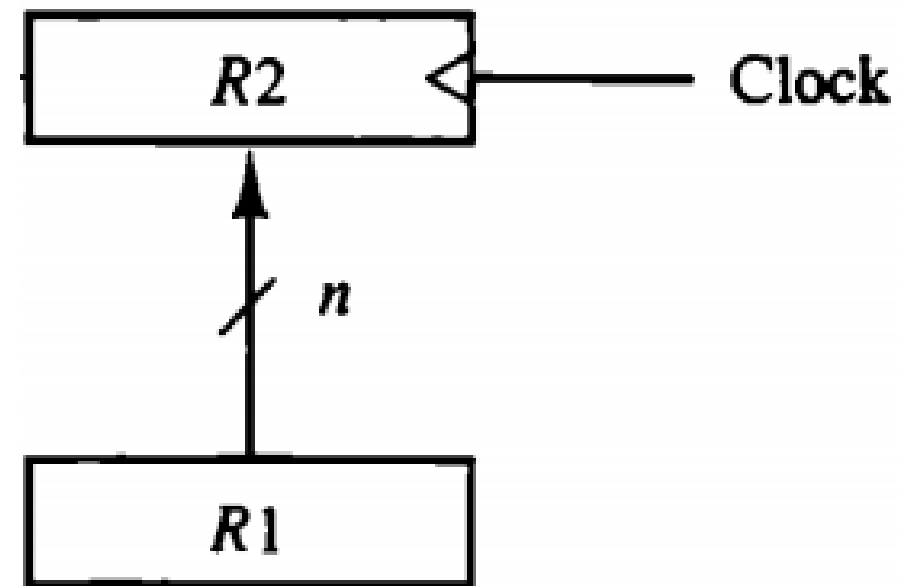
S0	S1	S2	Register
0	0	0	X
0	0	1	AR
0	1	0	PC
0	1	1	DR
1	0	0	AC
1	0	1	IR
1	1	0	TR
1	1	1	Memory



- A **digital computer** is best defined by specifying:
 - The **set of registers** it contains and their function.
 - The **sequence of microoperations** performed on the binary data in the registers.
 - The **control that initiates** the sequence of microoperations.
- The symbolic notion used to describe the microoperation transfers among registers is called a **Register Transfer Language (RTL)**

- A **transfer of content** of register R1 into register R2. the contents of the source R1 does not change:

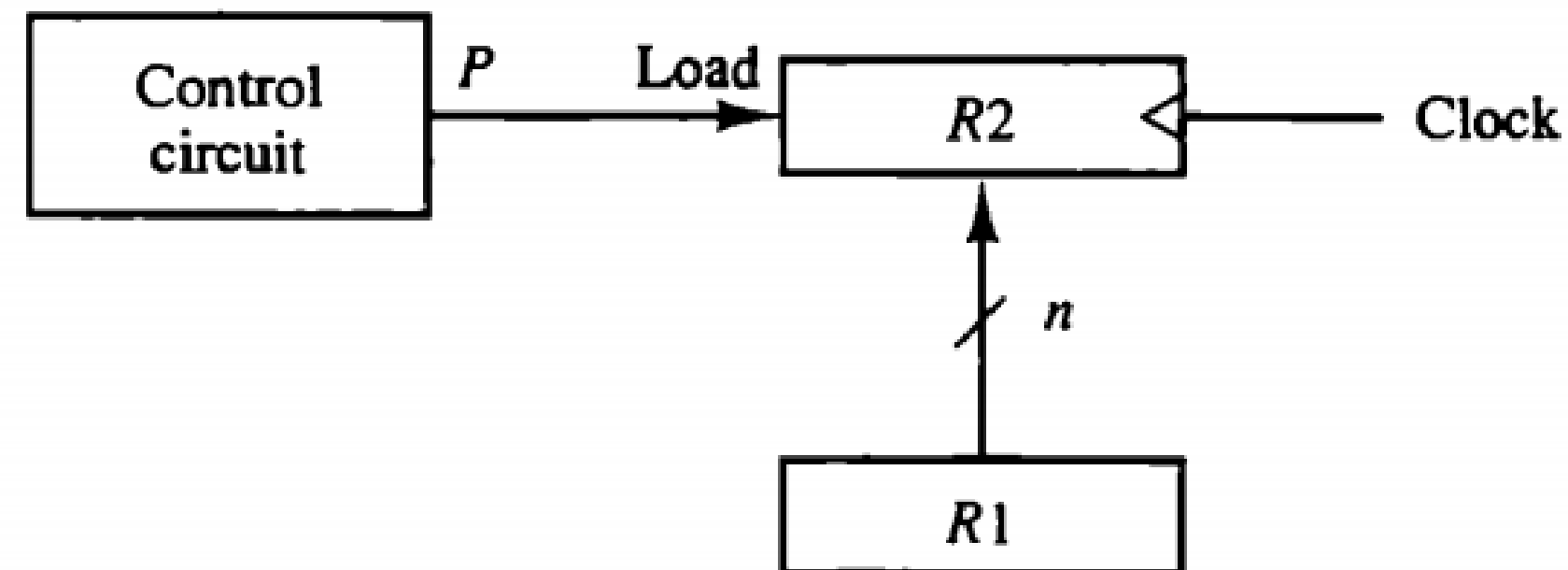
$R2 \leftarrow R1$



- We usually want this transfer to happen **under certain conditions**:

If ($P = 1$) then ($R2 \leftarrow R1$)

$P: R2 \leftarrow R1$



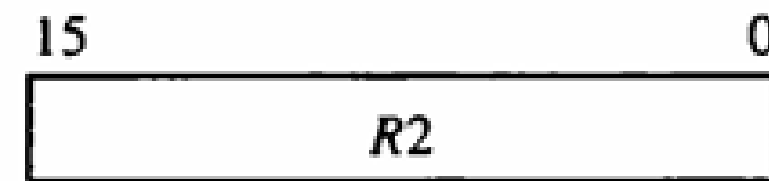
Register Transfer Language



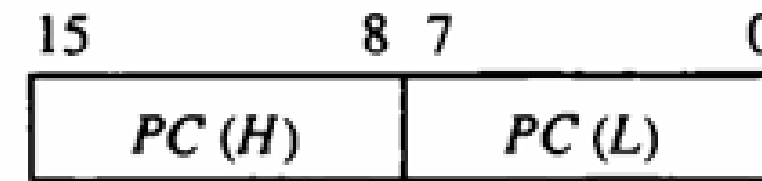
(a) Register R



(b) Showing individual bits



(c) Numbering of bits



(d) Divided into two parts

Symbol	Description	Examples
Letters (and numerals)	Denotes a register	$MAR, R2$
Parentheses ()	Denotes a part of a register	$R2(0-7), R2(L)$
Arrow $\leftarrow$	Denotes transfer of information	$R2 \leftarrow R1$
Comma ,	Separates two microoperations	$R2 \leftarrow R1, R1 \leftarrow R2$

- A bus contains a **set of lines**, one for each bit of a register, through which binary information is transferred **one at a time**. **Control signals** determine which register is selected during register transfer.

$BUS \leftarrow C$

$R1 \leftarrow BUS$

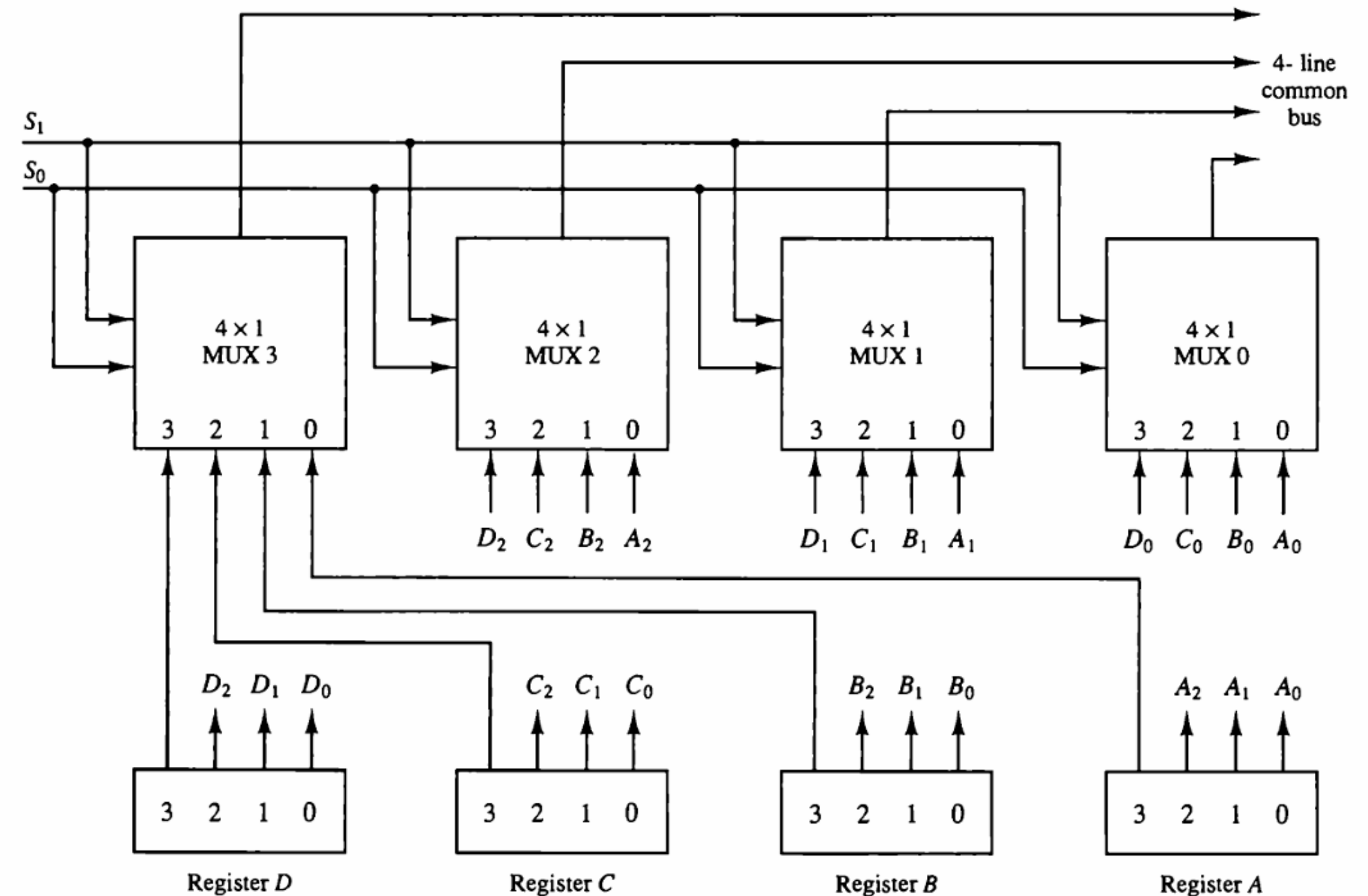


$R1 \leftarrow C$

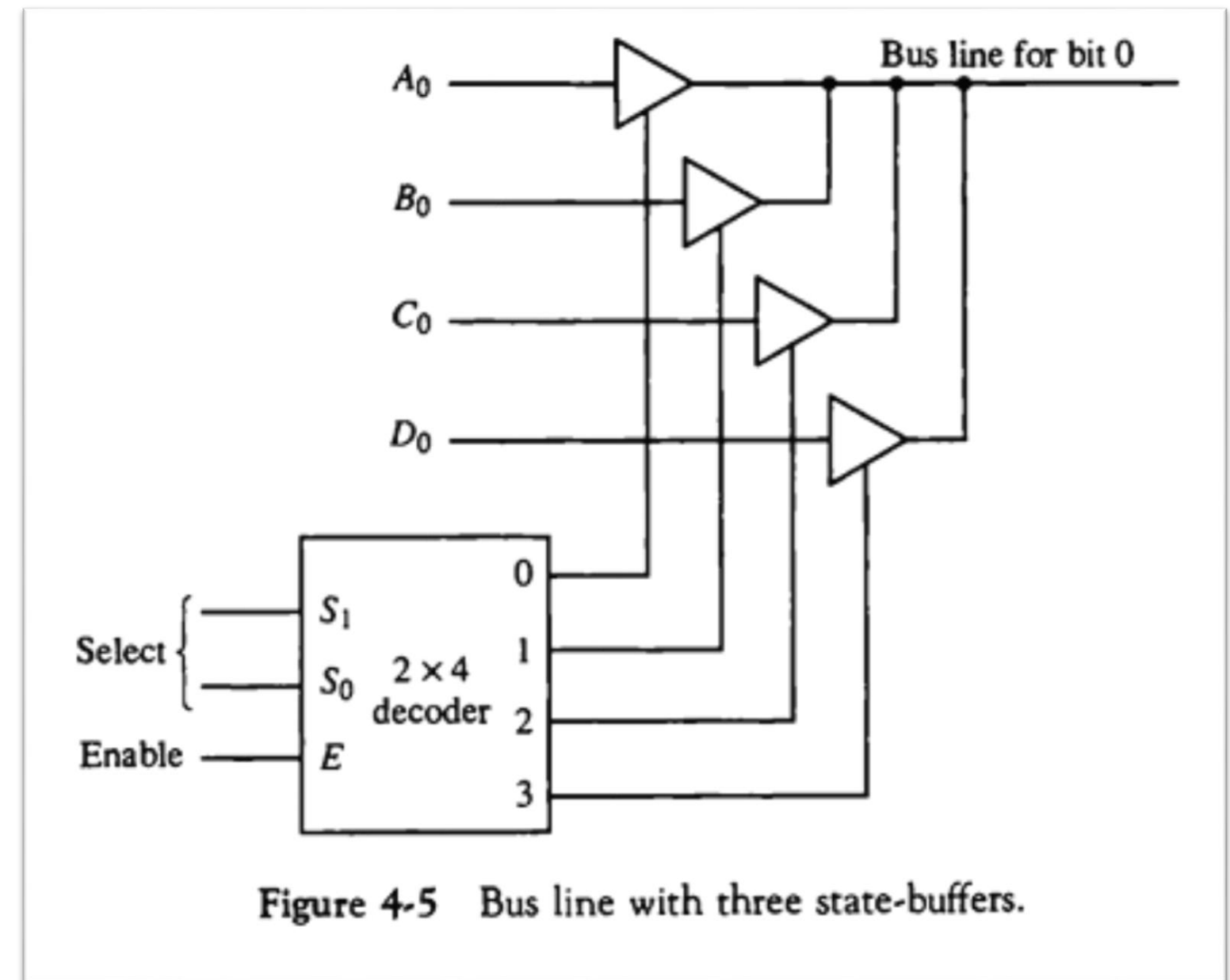
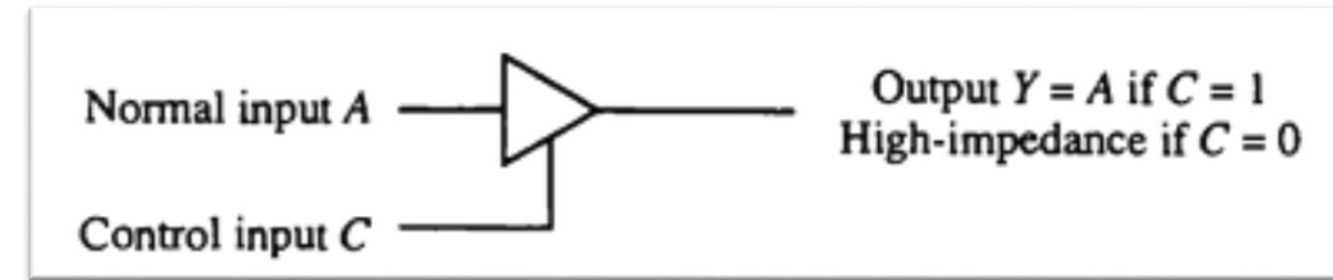
TABLE 4-2 Function Table for Bus of Fig. 4-3

S_1	S_0	Register selected
0	0	A
0	1	B
1	0	C
1	1	D

Figure 4-3 Bus system for four registers.



- One way to ensure only **one control** input is **active** at any given time is to use a **decoder**.
- When the enable input of the **decoder is 0**, all of its **outputs are 0**
- When the enable input is **active**, **one** of the buffers will be active, depending on the **select** input of the decoder.



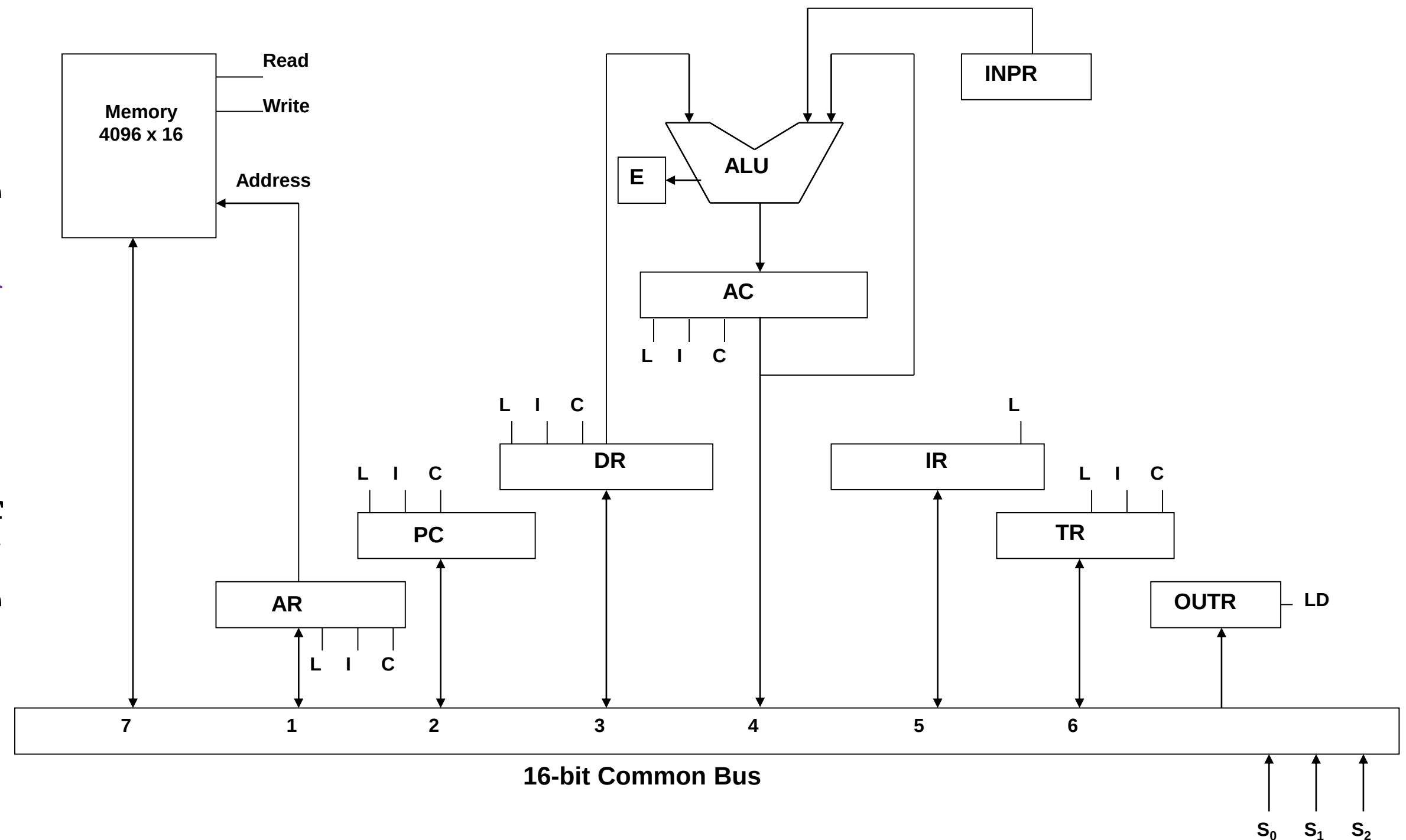
- The transfer of information **from a memory word to the outside** environment is called a **read** operation
- The transfer of new information to be **stored into the memory** is called a **write** operation.
- A memory word is symbolized by the letter M.
- It is necessary to specify the address of M in memory transfer operations.

Read: $DR \leftarrow M[AR]$

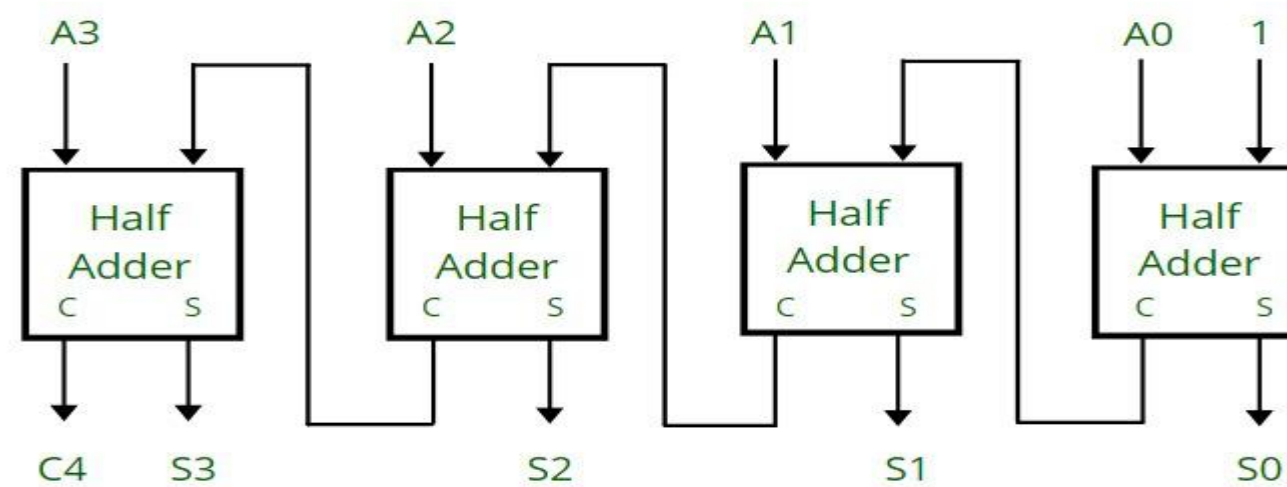
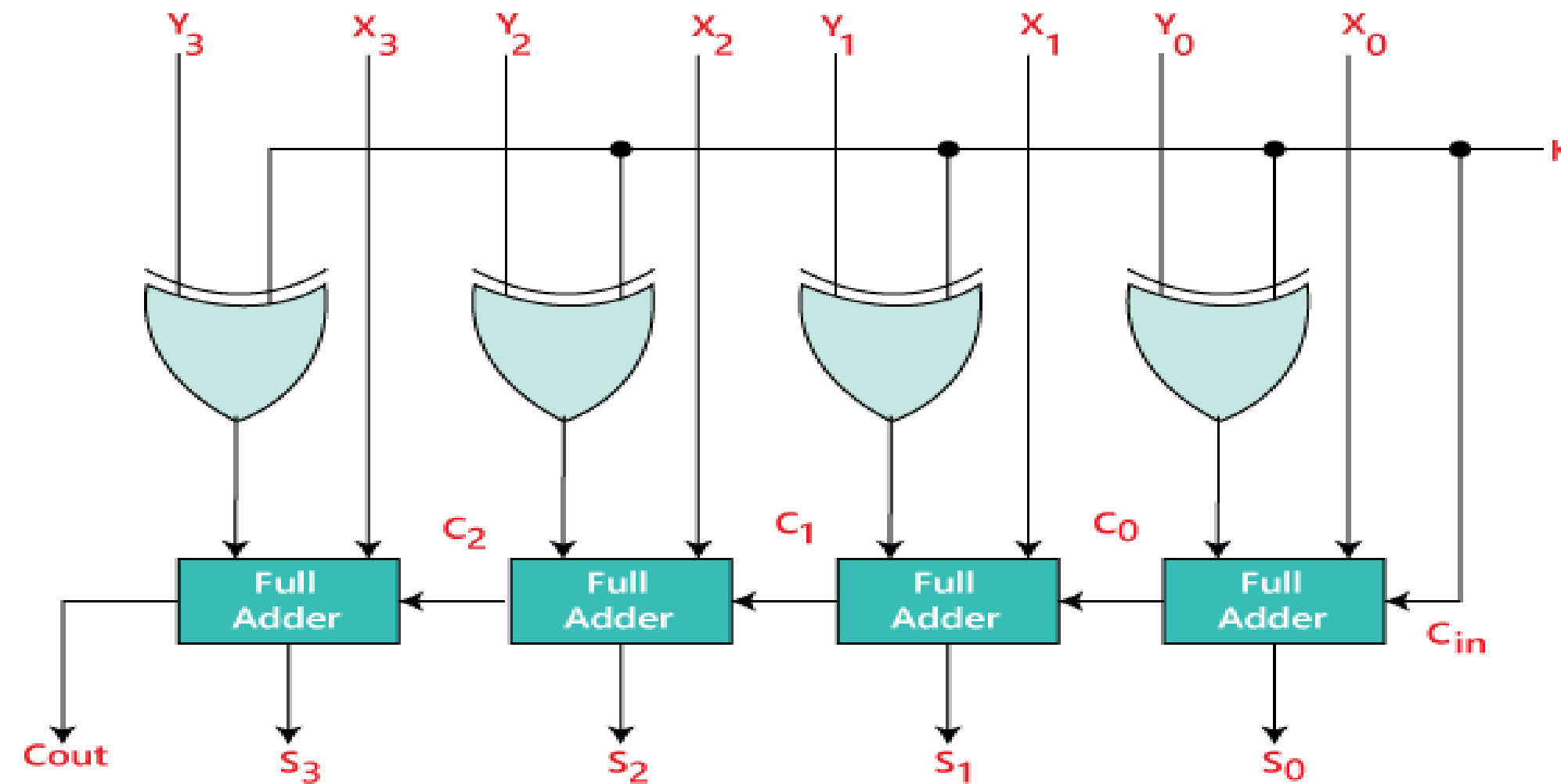
Write: $M[AR] \leftarrow R1$

Common Bus System

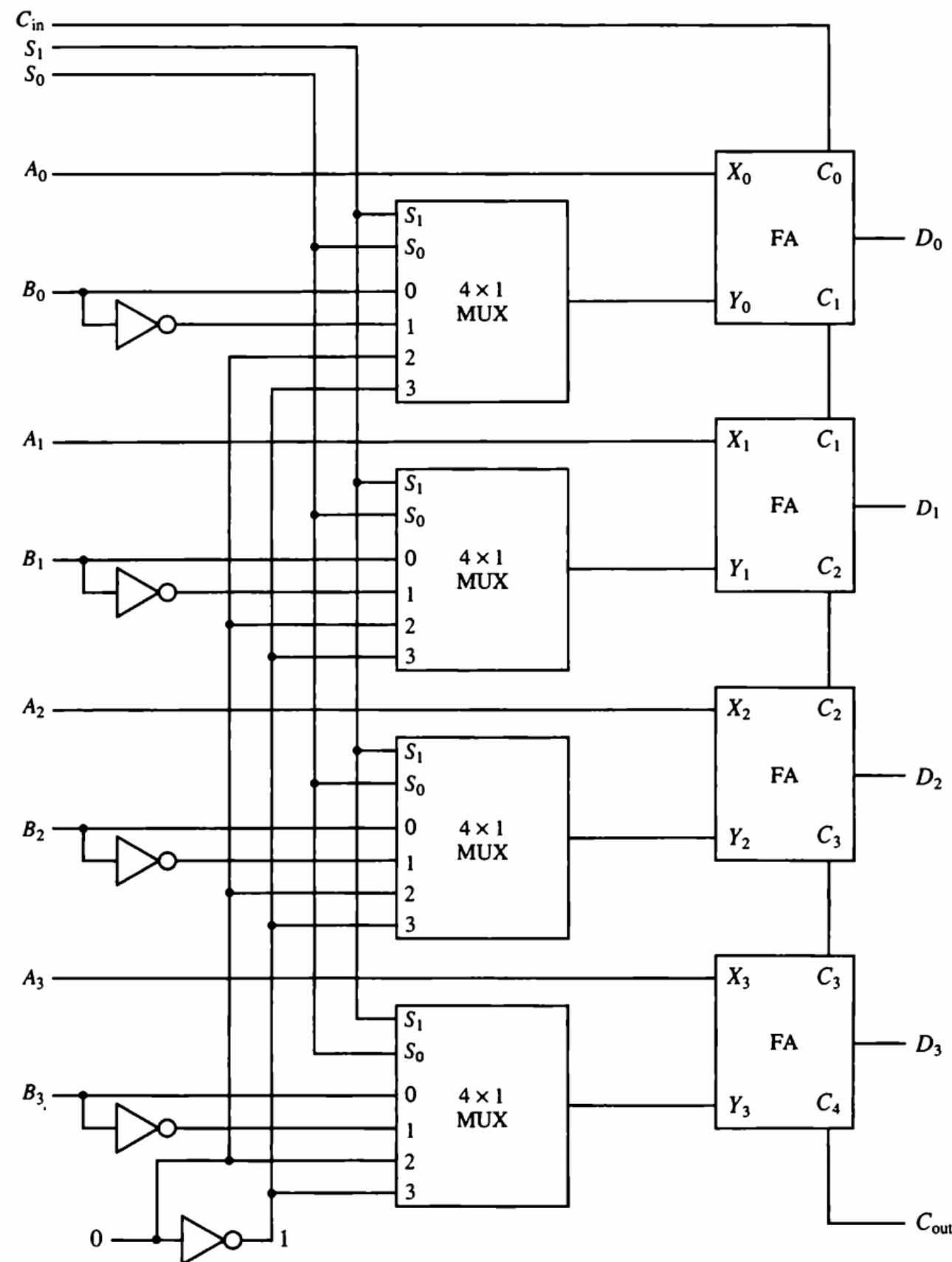
- Either one of the registers has its load signal activated or the memory has its read signal activated. This will determine where the data from the bus get loaded.
- The 12-bit registers have 0s loaded into their **high order 4 bits**.
- The **low order 8 bits** of the bus is loaded into the 8-bit OUTR register.



RTL – Binary ADD/SUB & INC



RTL – Binary Arithmetic Circuit

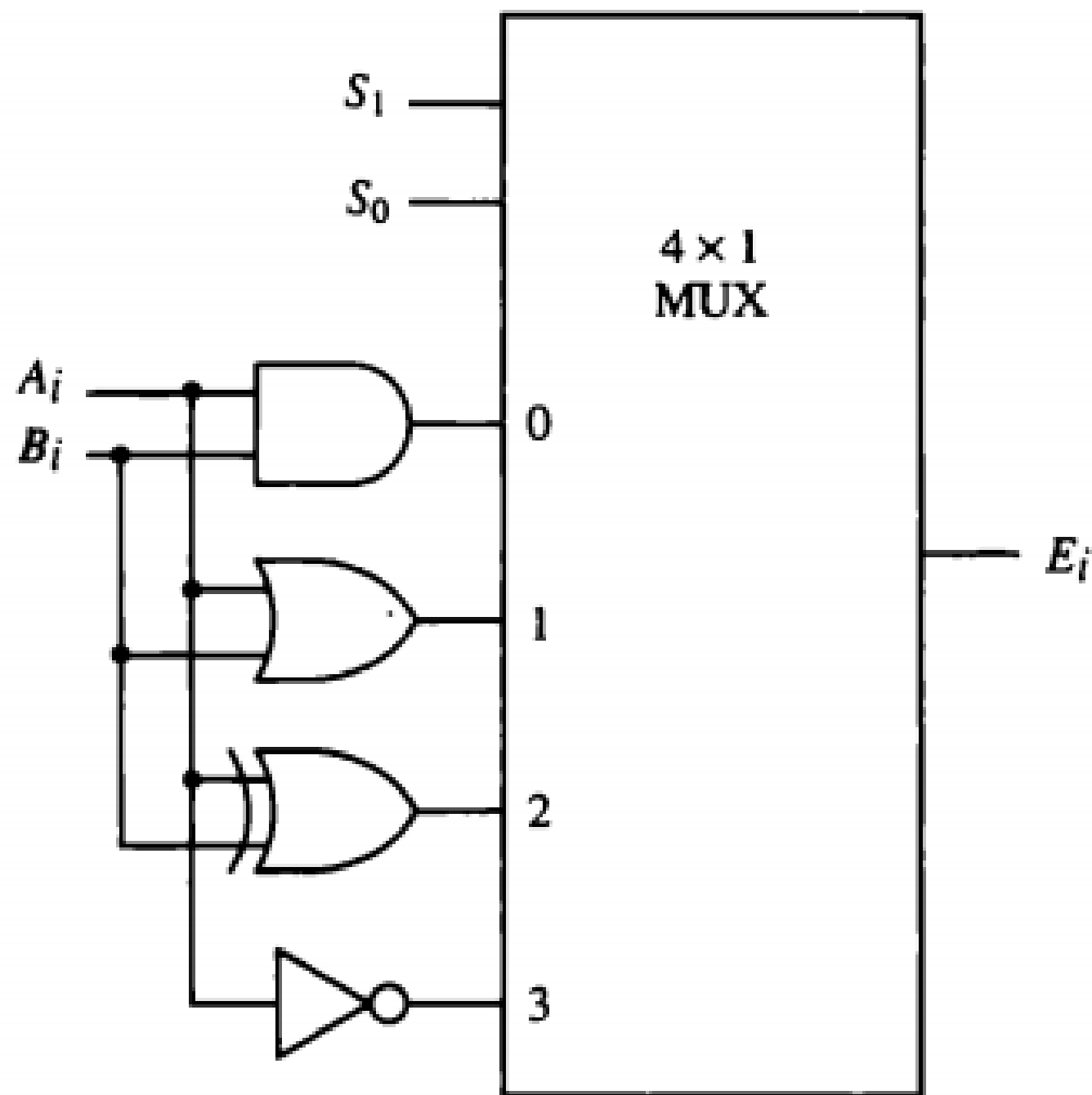


Select			Input Y	Output $D = A + Y + C_{in}$	Microoperation
S_1	S_0	C_{in}			
0	0	0	B	$D = A + B$	Add
0	0	1	B	$D = A + B + 1$	Add with carry
0	1	0	$\bar{B}$	$D = A + \bar{B}$	Subtract with borrow
0	1	1	$\bar{B}$	$D = A + \bar{B} + 1$	Subtract
1	0	0	0	$D = A$	Transfer A
1	0	1	0	$D = A + 1$	Increment A
1	1	0	1	$D = A - 1$	Decrement A
1	1	1	1	$D = A$	Transfer A

RTL- Binary Logic Functions and μop

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

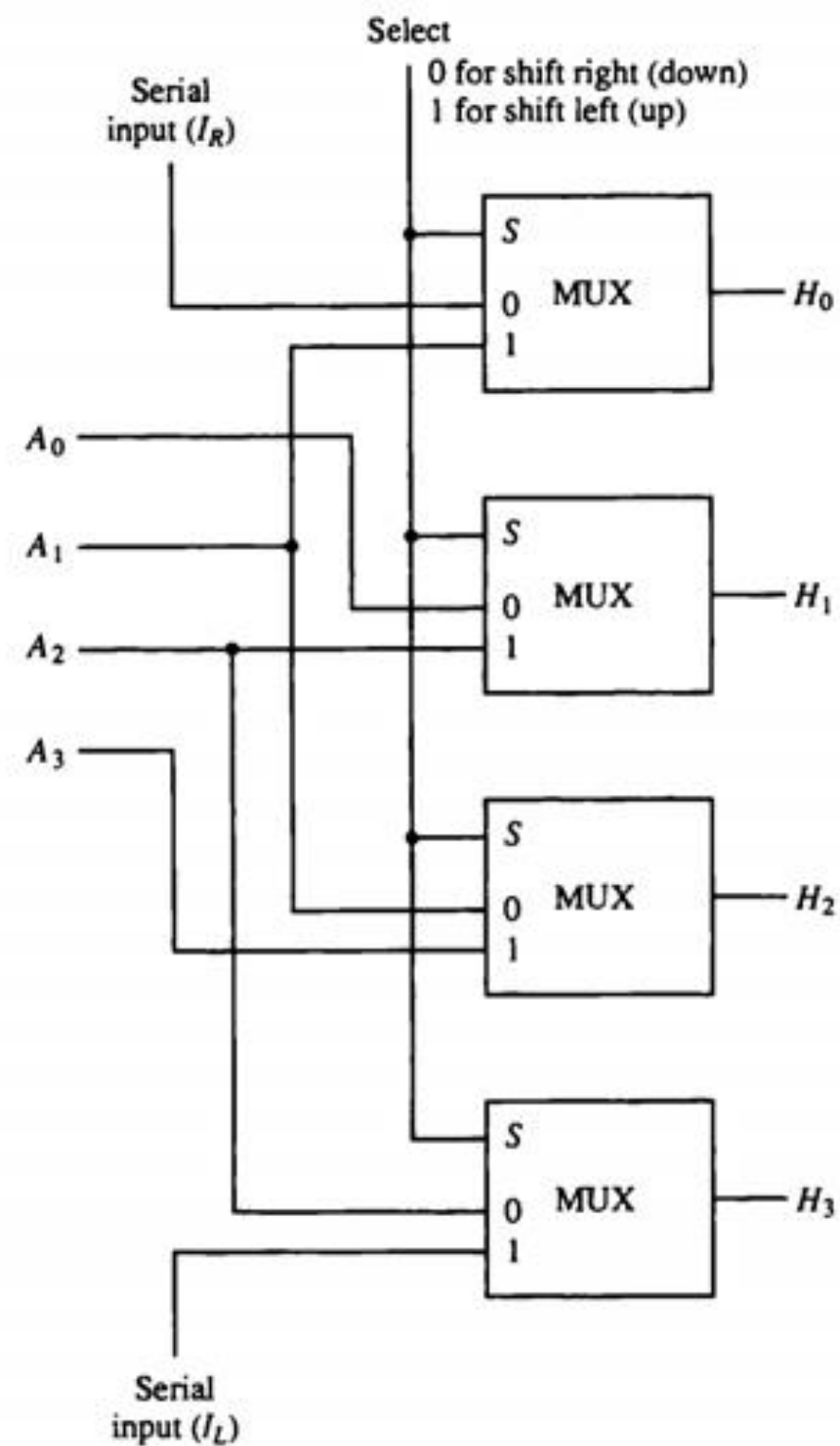
Boolean function	Microoperation	Name
$F_0 = 0$	$F \leftarrow 0$	Clear
$F_1 = xy$	$F \leftarrow A \wedge B$	AND
$F_2 = xy'$	$F \leftarrow A \wedge \overline{B}$	
$F_3 = x$	$F \leftarrow A$	Transfer A
$F_4 = x'y$	$F \leftarrow \overline{A} \wedge B$	
$F_5 = y$	$F \leftarrow B$	Transfer B
$F_6 = x \oplus y$	$F \leftarrow A \oplus B$	Exclusive-OR
$F_7 = x + y$	$F \leftarrow A \vee B$	OR
$F_8 = (x + y)'$	$F \leftarrow \overline{A \vee B}$	NOR
$F_9 = (x \oplus y)'$	$F \leftarrow \overline{A \oplus B}$	Exclusive-NOR
$F_{10} = y'$	$F \leftarrow \overline{B}$	Complement B
$F_{11} = x + y'$	$F \leftarrow A \vee \overline{B}$	
$F_{12} = x'$	$F \leftarrow \overline{A}$	Complement A
$F_{13} = x' + y$	$F \leftarrow \overline{A} \vee B$	
$F_{14} = (xy)'$	$F \leftarrow \overline{A \wedge B}$	NAND
$F_{15} = 1$	$F \leftarrow \text{all 1's}$	Set to all 1's



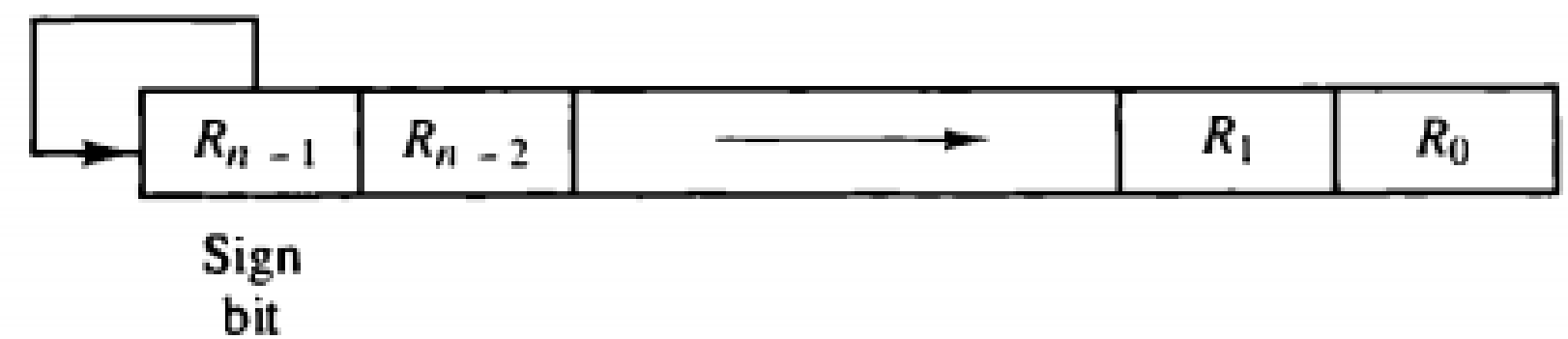
S_1	S_0	Output	Operation
0	0	$E = A \wedge B$	AND
0	1	$E = A \vee B$	OR
1	0	$E = A \oplus B$	XOR
1	1	$E = \bar{A}$	Complement

RTL - Binary Shift

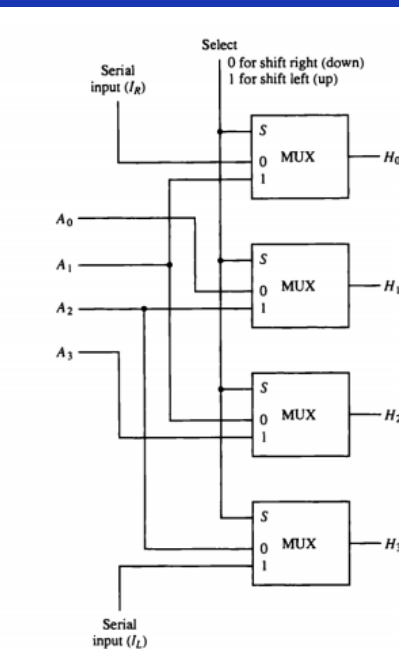
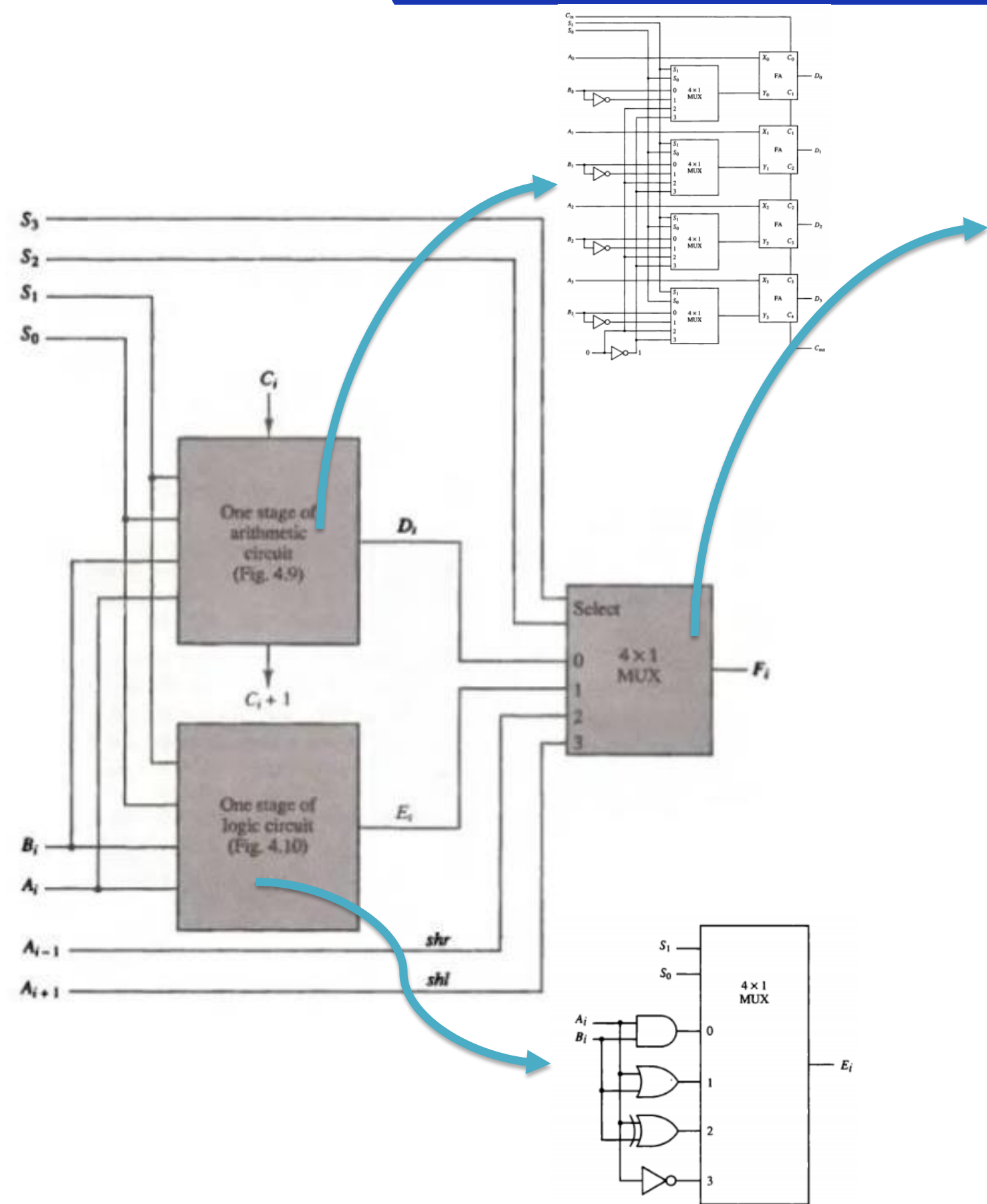
Select	Output			
S	H_0	H_1	H_2	H_3
0	I_R	A_0	A_1	A_2
1	A_1	A_2	A_3	I_L



Symbolic designation	Description
$R \leftarrow \text{shl } R$	Shift-left register R
$R \leftarrow \text{shr } R$	Shift-right register R
$R \leftarrow \text{cil } R$	Circular shift-left register R
$R \leftarrow \text{cir } R$	Circular shift-right register R
$R \leftarrow \text{ashl } R$	Arithmetic shift-left R
$R \leftarrow \text{ashr } R$	Arithmetic shift-right R



RTL – Arithmetic Logic Unit



Operation select					Operation	Function
S ₃	S ₂	S ₁	S ₀	C _{in}		
0	0	0	0	0	$F = A$	Transfer A
0	0	0	0	1	$F = A + 1$	Increment A
0	0	0	1	0	$F = A + B$	Addition
0	0	0	1	1	$F = A + B + 1$	Add with carry
0	0	1	0	0	$F = A + \overline{B}$	Subtract with borrow
0	0	1	0	1	$F = A + \overline{B} + 1$	Subtraction
0	0	1	1	0	$F = A - 1$	Decrement A
0	0	1	1	1	$F = A$	Transfer A
0	1	0	0	x	$F = A \wedge B$	AND
0	1	0	1	x	$F = A \vee B$	OR
0	1	1	0	x	$F = A \oplus B$	XOR
0	1	1	1	x	$F = \overline{A}$	Complement A
1	0	x	x	x	$F = \text{shr } A$	Shift right A into F
1	1	x	x	x	$F = \text{shl } A$	Shift left A into F



Thanks
